2주차 예습과제

📍 Transformer - Attention Is All You Need

📝 교재 개념정리

1. Introduction
    논문이 다루는 분야
    해당 task에서 기존 연구 한계점
    논문의 contributions
2. Related Work
3. 제안 방법론
    Main Idea
    Contribution
4. 실험 및 결과
    Dataset
    Baseline
    결과
5. 결론 (배운점)

1. Introduction

💡 논문에서 다루고 있는 주제가 무엇인지와 해당 주제의 필요성이 무엇인가
논문에서 제안하는 방법이 기존 방법의 문제점에 대응되도록 제안 되었는가

논문이 다루는 분야

- 순환 신경망(Recurrent neural network, RNN), 장단기 메모리(long short-term memory, LSTM), 게이트 순환 신경망(Gated RNN)은 시퀀스 모델링 및 변환(sequence modeling and transduction) 문제에서 최첨단(state-of-the-art) 접근 방법으로 자리잡아옴.
- 순환 기반 언어 모델(recurrent language models)과 인코더–디코더(encoder–decoder) 구조의 성능 한계를 확장하기 위한 다양한 연구들이 계속 진행되어옴.

해당 task에서 기존 연구 한계점

- 순환 모델은 일반적으로 입력 시퀀스와 출력 시퀀스의 기호 위치(symbol position)를 따라 계산을 분배
  - 계산 시간의 단계와 각 위치를 정렬, 이전 은닉 상태 h_{t-1}과 위치 t의 입력 기반으로 은닉 상태 h_t의 시퀀스를 생성 → 이전 hidden state에 의존하는 순차적 계산 구조
- 순차적인 특성 때문에 병렬화(parallelization)가 불가능
  - 시퀀스 길이가 길어질수록 메모리 제약 때문에 여러 예제를 동시에 처리하는 배치 크기 늘리는데 한계가 있어 더 중요한 문제
- 최근 연구에서는 factorization 기법, 조건부 계산(conditional computation)을 통해 계산 효율 및 모델 성능을 개선
- 그러나 **순차적 계산(sequential computation)**이라는 근본적인 제약은 여전히 남아 있음
- 어텐션 메커니즘
  - 강력한 시퀀스 모델링 및 변환 모델의 핵심 요소
  - 입력 또는 출력 시퀀스의 거리와 관계없이 의존성(dependencies, 관계)을 모델링 할 수 있도록 함
  - 대부분 순환 신경망과 함께 사용됨

논문의 contributions

- 새로운 모델 구조인 **Transformer**를 제안
  - 순환 구조를 배제 - 순환 신경망, 합성곱 신경망을 사용 X
  - 입력과 출력 사이의 전역적 의존성을 파악하기 위해 어텐션 메커니즘만을 기반으로 함
- Transformer는 훨씬 높은 수준의 병렬 처리를 가능하게 함
- 8개의 P100 GPU에서 12시간만의 학습만으로도 기계번역의 품질에서 SOTA 달성

2. Related Work

💡 Introduction에서 언급한 기존 연구들에 대해 어떻게 서술하는가
제안 방법의 차별성을 어떻게 표현하고 있는가

- 순차적 계산을 줄이려는 목표

- Extended Neural GPU, ByteNet, ConvS2S(Convolutional Sequence to Sequence)와 같은 모델의 기반이 됨
- 합성곱 신경망 기반, 입출력의 모든 위치에 대해 은닉 표현(hidden representation)을 병렬로 계산

  ❗ 두 개의 임의의 입력 도는 출력 위치 사이의 신호를 연결하기 위해 필요한 연산 수가 거리에 따라 증가 → 멀리 떨어진 위치 간의 의존성을 학습하기 어려워짐

⇒ Transformer에서는 필요한 연산 수가 상수(constant) 수준으로 줄어듬

- Attention은 여러 위치의 정보를 attention weight로 가중 평균하기 때문에 정보가 섞이면서 효과적인 해상도(effective resolution)가 감소하는 문제 발생

→ Multi-Head Attention을 통해 보완

- Self-attention (intra-attention)
  - 하나의 시퀀스 내부에서 서로 다른 위치들 사이의 관계를 계산하여 시퀀스의 표현을 생성하는 어텐션 방식
  - 독해 이해, 추상적 요약, 텍스트 함의, 작업 독립적인 문장 표현 학습 등에서 활용되어 옴
- End-to-end Memory Networks
  - 시퀀스 정렬 기반 순환(sequence-aligned recurrence) 대신 순환적 어텐션 메커니즘(recurrent attention mechanism)을 기반으로 함
  - 단순 언어 질의응답, 언어 모델링 작업에서 좋은 성능 보임

⇒ Transformer는 입력과 출력 표현 계산할 때 self-attention에만 의존하는 최초의 시퀀스 변환 모델

- sequence-aligned RNN 또는 convolution을 사용하지 X

3. 제안 방법론

💡 Introduction에서 언급된 내용과 동일하게 작성되어 있는가
Introduction에서 언급한 제안 방법이 가지는 장점에 대한 근거가 있는가
제안 방법에 대한 설명이 구현 가능하도록 작성되어 있는가

Main Idea

- 신경망 기반 시퀀스 변환 모델은 인코더-디코더 구조를 사용
  - 인코더: 입력 시퀀스의 기호 표현(x_1, ..., x_n)을 연속적인 벡터 표현 z=(z_1, ..., z_n)로 변환
  - 디코더는 이 z를 기반으로 출력 시퀀스(y_1, ..., y_m)을 한 토큰씩 생성
  - 자기회귀(auto-regressive) 방식으로 동작 → 이전에 생성한 토큰을 다음 예측의 입력으로 사용
- Transformer도 인코더-디코더 구조를 사용
  - 이때 내부 연산을 stacked self-attention과 point-wise, fully connected(feed forward) layer로 구성

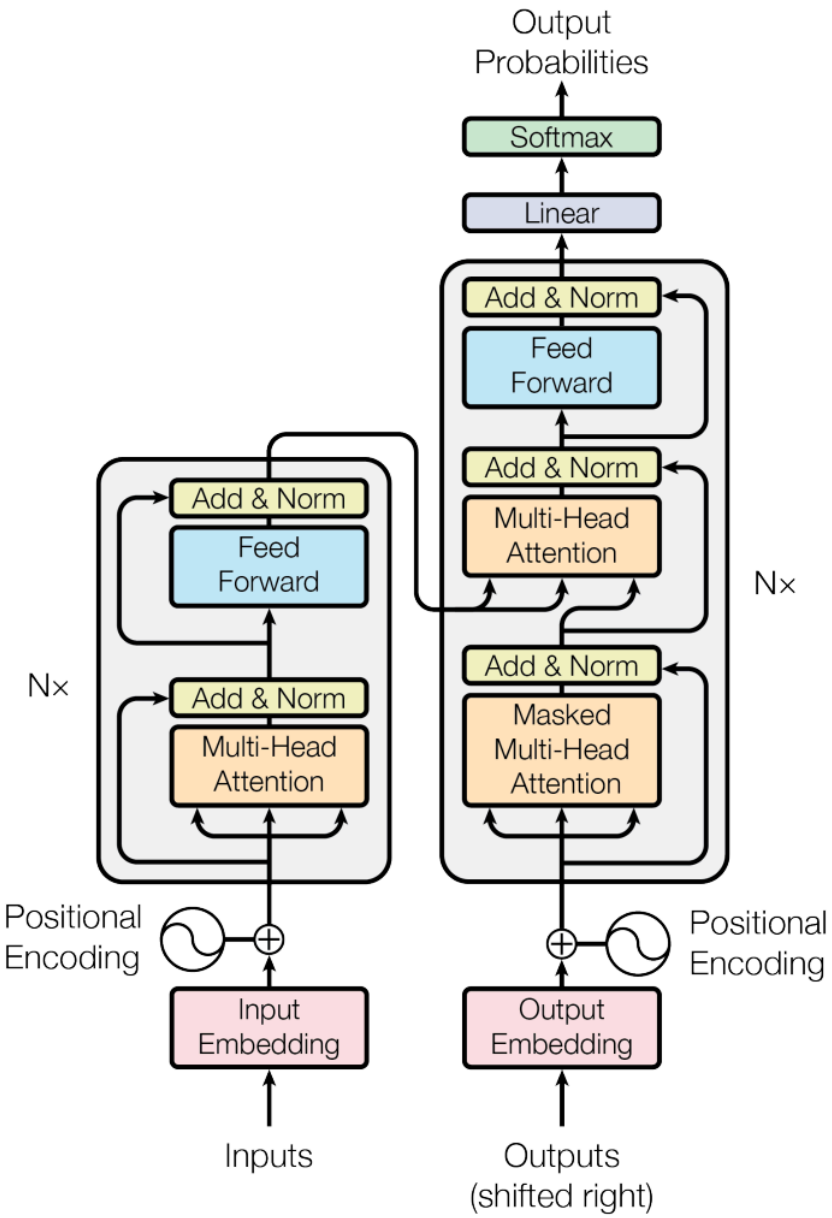

Figure 1: The Transformer - model architecture.

3.1 Encoder and Decoder Stacks

- **Encoder:**
  - N = 6개의 동일한 층이 쌓인 구조
  - 각 층은 두 개의 sub-layer로 구성
    - multi-head self-attention
    - position-wise fully connected feed -forward network
  - 각 서브 레이어에 residual connection을 적용, layer normalization 수행
    - 출력: $LayerNorm(x + Sublayer(x))$

    - 모든 서브 레이어와 임베딩 레이어는 d_{model} = 512의 출력 생성
- **Decoder:**
    - N = 6개의 동일한 층이 쌓인 구조
    - 각 층은 인코더 층에서 세 번째 서브 레이어를 추가한 구성
        - 인코더 출력에 multi-head attention을 수행
    - 각 서브 레이어에 residual connection을 적용, layer normalization 수행
    - 디코더의 self-attention은 미래 위치를 참조하지 못하도록 되어있음
        - 마스킹을 적용해 현재 위치가 미래 토큰들을 보지 못하게 함
        - 출력 임베딩이 한 위치씩 오른쪽으로 이동되어 있는 구조와 결합

          →위치 i의 예측이 반드시 i보다 이전 위치의 출력 정보만을 사용하도록 보장

3.2 Attention

- Attention 함수
    - 하나의 query와 여러 개의 key-value 쌍을 입력으로 받아 출력 벡터를 생성하는 함수 (query, key, value, 출력은 모두 벡터)
    - 출력은 value 벡터들의 가중합(weighted sum)으로 계산
    - 각 value에 부여되는 가중치는 query와 해당 key 사이의 호환성 함수를 통해 계산

      → query와 key가 얼마나 관련 있는지를 특정해 그 결과를 value에 적용할 가중치로 사용

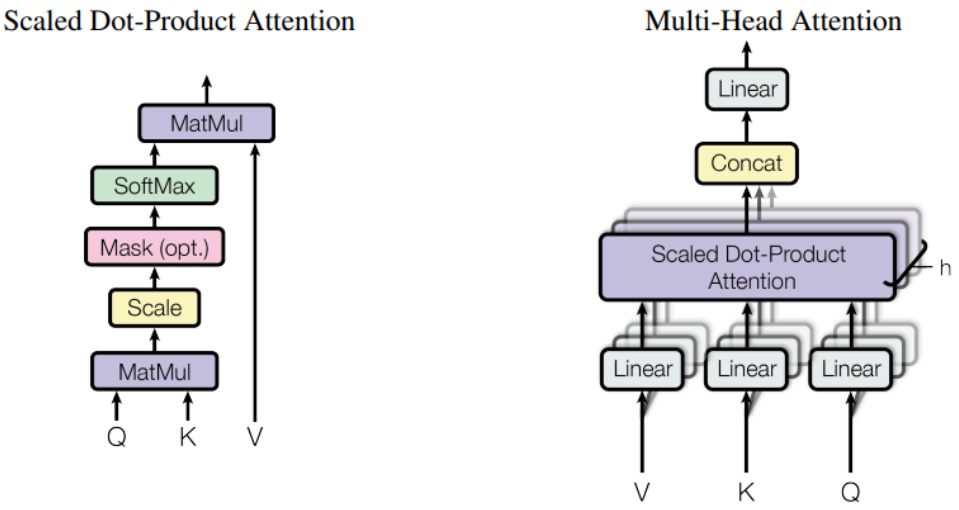

Figure 2: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel.

3.2.1 Scaled Dot-Product Attention

- 입력: qurey, key(차원 d_k), value(차원 d_v)
- query와 모든 key 사이의 내적(dot product)계산 → $\sqrt{d_k}$로 나눔 → softmax 적용

  ⇒ 각 value에 대한 가중치

📌

- query를 행렬 Q, key와 value를 행렬 K와 행렬 V로 묶어 계산
  $Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$
- dot product 값을 $\sqrt{d_k}$로 나누어 스케일링을 적용 → softmax에서 gradient가 너무 작아지는 문제를 방지

- 널리 사용되는 기존 attention 방식
  - Additive attention (FFN으로 유사도 계산)
    - 단일 은닉층을 가진 Feed Forward Network를 사용해 query와 key 사이의 호환성 함수를 계산
  - Dot-product attention (내적으로 계산)
    - 스케일링 계수 $\frac{1}{\sqrt{d_k}}$를 제외하면 Scaled Dot-Product Attention과 동일

- dot-product attention은 행렬 곱으로 구현 가능하기 때문에 훨씬 빠르고 메모리 효율적
- d_k가 커질 경우 dot-product attention보다 additive attention이 더 좋은 성능을 보이는 경향
  - d_k가 커질수록 dot product 값의 크기가 커져 softmax 함수가 매우 작은 gradient를 가지게 되기 때문 → scaling 적용해 해결

3.2.2 Multi-Head Attention

- query, key, value를 여러 개의 서로 다른 선형 변환(linear projection)으로 나누어 처리하는 방식
- 각 projection에서 query, key, value에 대해 attention 연산을 병렬로 수행, 출력된 여러 개의 attention 결과(head) 연결(concatenate)하고 다시 한 번 선형 변환을 적용해 최종 출력을 얻음
- multi-head attention은 모델이 여러 표현 공간(representation subspace)에서 다양한 관계를 동시에 학습할 수 있도록 함

📌 $MultiHead(Q, K, V) = Concat(head_1, ..., head_h)W^O$
$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$

- 각 projection 행렬은 다음과 같은 차원을 가진다.
  - $W_i^Q \in R^{d_{model} \times d_k}$
  - $W_i^K \in R^{d_{model} \times d_k}$
  - $W_i^V \in R^{d_{model} \times d_v}$
  - $W^O \in R^{hd_v \times d_{model}}$

- 논문에서는 h=8개의 병렬 attention head를 사용, 각 head에 대해 $d_k = d_v = d_{model}/h = 64$

  → head의 차원이 줄어들어 연산 비용이 단일 attention과 거의 비슷
- Trasformer는 Multi-Head Attention을 세 가지 방식으로 사용
  - Encoder-Decoder Attention
    - query는 이전 디코더 레이어의 출력에서, key와 value는 인코더의 출력에서

        - 디코더의 각 위치가 입력 시퀀스의 모든 위치 참조 가능
    - Encoder의 Self-Attention
        - key, value, query가 모두 같은 입력에서 생성, 이전 인코더 레이어의 출력이 사용됨

          → encoder 내부에서 모든 토큰이 서로를 참조
    - Decoder의 Self-Attention
        - 현재 위치는 이전 토큰들만 볼 수 있도록 scaled dot-product attention 내부에서 masking을 적용

          →auto-regressive 방식(이전 토큰 기반 생성)을 유지할 수 있음

3.3 Position-wise Feed-Forward Networks

- 인코더와 디코더에는 attention 이후 FFN(fully connected feed-forward network)이 위치
- FFN은 각 토큰 위치에 독립적으로 동일하게 적용되는 구조
    - 두 개의 선형 변환과 그 사이에 위치한 ReLU 활성화 함수로 구성 (Linear → ReLU → Linear)

📌 $FFN(x) = max(0, xW_1 + b_1)W_2 + b_2$

    - 선형 변환 자체는 모든 위치에 동일하게 적용되지만, 레이어마다 서로 다른 파라미터(parameter) 사용
- kernel size = 1인 두 개의 convolution으로도 해석 가능
- 입력/출력 차원은 512, 내부 hidden layer는 2048

3.4 Embeddings and Softmax

- 학습된 임베딩을 사용해 입력 토큰과 출력 토큰을 d_{model}차원의 벡터로 변환
- 디코더의 출력은 선형 변환과 softmax를 통해 다음 토큰 확률로 변환됨
- 두 개의 임베딩 레이어(입력 임베딩, 출력 임베딩)와 softmax 이전 선형 변환 사이에서 동일한 가중치 행렬을 공유
- 임베딩 레이어에서는 이 가중치에 $\sqrt{d_{model}}$을 곱해 사용

→ 가중치 공유로 파라미터 수를 줄이고 모델 성능을 향상시키는 효과가 있다.

Table 1: Maximum path lengths, per-layer complexity and minimum number of sequential operations for different layer types. n is the sequence length, d is the representation dimension, k is the kernel size of convolutions and r the size of the neighborhood in restricted self-attention.

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

3.5 Positional Encoding

- 순환 구조나 합성곱을 사용 X → 시퀀스의 순서 정보를 활용하려면 토큰의 위치에 대한 정보를 별도로 제공해야 함

  → 인코더와 디코더 입력 임베딩에 위치 인코딩(positional encoding)을 더해줌
- positional encoding은 임베딩과 동일한 차원을 가지므로 임베딩 벡터와 더해서 사용 가능
- positional encoding을 만드는 방법에는 학습 가능한 방식(learned positional encoding)과 고정된 방식(fixed positional encoding)등 여러 방식이 존재
- 논문에서는 sin, cos 함수 기반의 sinusoidal positional encoding을 사용

📌

- 각 차원은 서로 다른 주파수의 사인/코사인 파형으로 구성되어 위치 정보를 표현

$$PE(pos, 2i) = sin(pos/10000^{2i/d_{model}})$$
$$PE(pos, 2i+1) = \cos(pos/10000^{2i/d_{model}})$$

- pos는 토큰의 위치(position), i는 차원을 의미
- positional encoding의 각 차원은 하나의 사인파(sinusoid)에 해당
  - 이때 파장의 길이는 2π에서 10000·2π까지 기하급수적으로 증가

- 모델이 상대적 위치를 쉽게 학습 가능, 더 긴 시퀀스 길이에 대해서도 모델이 일반화 가능

Contribution

4. Why Self-Attention

- self-attention, RNN, CNN을 비교
  1. 레이어당 총 계산 복잡도
  2. 얼마나 많은 계산을 병렬화할 수 있는가 → 필요한 최소 순차 연산의 수로 측정
  3. 장거리 의존성(long-range dependencies) 사이의 경로 길이
     - long-range dependency 학습은 시퀀스 변환 문제에서 매우 중요
     - 두 위치 사이의 경로가 짧을수록 모델이 장거리 관계를 더 쉽게 학습할 수 있음

       → 각 구조에서 두 위치 사이의 최대 경로 길이도 함께 비교
- Self-Attention vs. RNN
  - 순차 연산 개수 - self-attention: O(1), RNN: O(n)
  - 계산 복잡도 측면에서 보통 문장 길이 n < 표현 차원 d이기 때문에 self-attention이 recurrent보다 빠름
  - 긴 시퀀스에서 self-attention을 출력 위치 주변의 크기 r인 이웃으로 제한하면 계산량 감소 가능 → 최대 경로 길이(maximum path length)는 O(n/r)로 증가
- vs. CNN
  - CNN은 모든 위치를 연결하려면 여러 층이 필요하여 path length가 길어진다
  - CNN의 계산 비용을 줄여도 self-attention+point-wise feed-forward를 결합한 구조와 동일한 수준이 됨
- self-attention은 서로 다른 문법/의미 역할을 학습, 모델의 해석 가능성을 높일 수 있음

4. 실험 및 결과

💡 Introduction에서 언급한 제안 방법의 장점을 검증하기 위한 실험이 있는가

Dataset

5.1 Training Data and Batching

- WMT 2014 English–German 데이터셋
  - 약 450만 개의 문장 쌍으로 이루어짐

    - Byte-Pair Encoding (BPE) 사용해 문장 인코딩
    - source와 target이 공유하는 약 37,000개의 토큰 vocabulary를 사용
- WMT 2014 English–French 데이터셋
    - 3,600만 개의 문장으로 구성
    - 토큰은 32,000개의 word-piece vocabulary로 분할
- training batch
    - 약 25,000개의 source 토큰과 25,000개의 target 토큰을 포함하는 문장 쌍들의 집합으로 구성

5.2 Hardware and Schedule

- 8개의 NVIDIA P100 GPU
- base 모델의 하이퍼파라미터 사용한 경우 각 학습 step은 약 0.4초 소요, 총 100,000 step (약 12시간) 동안 학습
- big 모델의 경우 step 당 약 1.0초 소요, 300,000 step (약 3.5일) 동안 학습

5.3 Optimizer

- Adam optimizer 사용
    - β1=0.9, β2=0.98, ϵ=10^{-9}
    - $lrate = d_{model}^{-0.5} \cdot \min(step_num^{-0.5},\ step_num \cdot warmup_steps^{-1.5})$

      → 초기 warmup 단계에서는 lr을 선형적으로 증가, 이후에는 step 수의 역제곱근에 비례해 감소 (warmup steps = 4000)

5.4 Regularization

- 세 가지 정규화 기법 사용
    - Residual Dropout (dropout = 0.1)
        - dropout을 각 sub-layer의 출력에 적용 후 입력과 더하고(normalization 이전) 정규화

        - encoder와 decoder에서 임베딩과 positional encoding을 더한 결과에도 적용
    - Label Smoothing
        - 훈련 중 label smoothing을 사용(ϵ_{ls} = 0.1)
        - 모델이 정답을 지나치게 확신하지 않도록 만듦

          → perplexity는 나빠질 수 있지만 accuracy, BLEU score는 향상

Baseline

- Transformer의 성능 평가 위해 기존 모델들과 비교 실험 진행

1. Machine Translation Baselines
    - **ByteNet**
        - Dilated Convolution 기반 sequence-to-sequence 모델
        - CNN 기반 번역 모델
    - **Deep-Att + PosUnk**
        - Attention 기반 NMT 모델
        - unknown token 처리(PosUnk)
    - **GNMT + RL**
        - Google Neural Machine Translation
        - LSTM 기반 encoder–decoder
        - Reinforcement Learning 적용
    - **ConvS2S**
        - Convolutional Sequence-to-Sequence
        - CNN 기반 번역 모델
        - 병렬 연산 가능
    - **MoE (Mixture of Experts)**
        - 여러 expert network 조합
        - 입력별 expert 선택
    - **Ensemble Models**

    - Deep-Att + PosUnk Ensemble
    - GNMT + RL Ensemble
    - ConvS2S Ensemble

2. English Constituency Parsing Baselines
   - **BerkeleyParser**
     - Probabilistic parsing model
     - Penn Treebank 기반
   - **RNNG (Recurrent Neural Network Grammar)**
     - RNN 기반 문법 모델
     - 높은 parsing 성능
   - **Seq2Seq Parsing Models**
     - RNN 기반 sequence-to-sequence
     - parsing task 적용

결과

6.1 Machine Translation

- Transformer big 모델
  - WMT 2014 English-to-German 번역 작업
    - 기존 모델들보다 2.0 BLEU 이상 높은 성능 기록(28.4 BLEU)
    - base모델도 훨씬 적은 학습 비용으로 기존보다 높은 성능 달성
  - WMT 2014 English-to-French 번역 작업
    - BLEU 41.0을 기록
    - 기존 최고 모델보다 학습 비용이 1/4 이하
- 추론 단계에서는 beam search 사용
  - beam size = 4, length penalty α=0.6
  - 최대 출력 길이는 입력 길이 +50으로 설정
  - training time × GPU 수 × GPU FLOPs를 이용해 총 연산량(FLOPs) 추정

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	**41.29**	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$\mathbf{3.3 \cdot 10^{18}}$	
Transformer (big)	**28.4**	**41.8**	$2.3 \cdot 10^{19}$	

6.2 Model Variations

Table 3: Variations on the Transformer architecture. Unlisted values are identical to those of the base model. All metrics are on the English-to-German translation development set, newstest2013. Listed perplexities are per-wordpiece, according to our byte-pair encoding, and should not be compared to per-word perplexities.

	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	train steps	PPL (dev)	BLEU (dev)	params $\times 10^6$
base	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65
(A)				1	512	512				5.29	24.9	
				4	128	128				5.00	25.5	
				16	32	32				4.91	25.8	
				32	16	16				5.01	25.4	
(B)					16					5.16	25.1	58
					32					5.01	25.4	60
(C)	2									6.11	23.7	36
	4									5.19	25.3	50
	8									4.88	25.5	80
		256			32	32				5.75	24.5	28
		1024			128	128				4.66	26.0	168
			1024							5.12	25.4	53
			4096							4.75	26.2	90
(D)							0.0			5.77	24.6	
							0.2			4.95	25.5	
								0.0		4.67	25.3	
								0.2		5.47	25.7	
(E)	positional embedding instead of sinusoids									4.92	25.7	
big	6	1024	4096	16			0.3		300K	**4.33**	**26.4**	213

- Transformer의 구성 요소 분석 실험
  - newstest2013에서 수행(English-to-German 번역 작업의 development set)

- Attention head 수 실험
  - single-head attention에서는 성능 감소(-0.9 BLEU)
  - attention head 수가 지나치게 많아도 성능이 감소하는 경향
- Ket dimension d_k 크기 실험
  - d_k를 줄이면 성능 감소
  - query-key 유사성(compatibility) 계산의 역할이 중요함 → 단순한 dot-product보다 더 복잡한 compatibility function이 효과적일 수 있음
- 모델 크기와 정규화
  - 모델 크기가 커질수록 성능이 좋아짐
  - dropout이 overfitting을 방지하는 데 매우 효과적
- Positional encoding 비교
  - sinusoidal positional encoding과 learned positional embedding은 성능이 거의 동일

6.3 English Constituency Parsing

- Transformer가 번역 외 다른 작업에도 일반화될 수 있는지 평가하기 위해 실험
- 출력이 강한 구조적 제약 가짐, 출력 길이가 입력보다 훨씬 길, 데이터가 적은 상황에서 기존의 RNN 기반 sequence-to-sequence 모델이 sota 성능을 달성하지 못했단 문제가 있음
- Penn Treebank의 Wall Street Journal (WSJ)을 사용해 학습
  - 약 4만 개의 학습 문장을 포함
  - 16K 토큰 vocabulary
- semi-supervised 설정에서도 모델을 학습, 이 경우 high-confidence corpus와 BerkeleyParser corpus를 함께 사용
  - 약 1,700만 개의 문장으로 구성
  - 32K 토큰 vocabulary
- 실험 설정
  - 4-layer Transformer (d_model = 1024), dropout, learning rate, beam size 조정해 실험

- 추론 단계에서는 최대 출력 길이를 입력 길이 +300으로 증가, beam size = 21, length penalty α=0.3으로 설정

Table 4: The Transformer generalizes well to English constituency parsing (Results are on Section 23 of WSJ)

Parser	Training	WSJ 23 F1
Vinyals & Kaiser el al. (2014) [37]	WSJ only, discriminative	88.3
Petrov et al. (2006) [29]	WSJ only, discriminative	90.4
Zhu et al. (2013) [40]	WSJ only, discriminative	90.4
Dyer et al. (2016) [8]	WSJ only, discriminative	91.7
Transformer (4 layers)	WSJ only, discriminative	91.3
Zhu et al. (2013) [40]	semi-supervised	91.3
Huang & Harper (2009) [14]	semi-supervised	91.3
McClosky et al. (2006) [26]	semi-supervised	92.1
Vinyals & Kaiser el al. (2014) [37]	semi-supervised	92.1
Transformer (4 layers)	semi-supervised	92.7
Luong et al. (2015) [23]	multi-task	93.0
Dyer et al. (2016) [8]	generative	93.3

- 특별한 task-specific 튜닝 없이도 대부분의 기존 모델보다 높은 성능 달성(RNNG 제외)

  ⇒ Transformer는 번역뿐 아니라 구조적 NLP 작업(parsing)에서도 강력한 성능을 보이며 적은 데이터 환경에서도 RNN 기반 모델보다 우수할 수 있음

5. 결론 (배운점)

💡 연구의 의의 및 한계점, 본인이 생각하는 좋았던/아쉬웠던 점 (배운점)

- Transformer의 핵심 기여
  - RNN, CNN 없이 오직 attention만으로 구성된 최초의 sequence transduction 모델.
  - multi-head self-attention이 encoder–decoder 구조의 핵심 연산
    - 문장 내 모든 토큰 간의 관계를 동시에 고려, 장거리 의존성(long-range dependency)을 효과적으로 학습
- 성능과 학습 속도
  - 병렬 연산 가능, RNN / CNN 기반 모델보다 훨씬 빠르게 학습 가능
  - WMT 2014 번역 task에서 새로운 state-of-the-art 달성
- 이후 자연어 처리 분야에서 Transformer 구조가 널리 활용되며 BERT, GPT와 같은 다양한 모델의 기반이 됨

- 향후 연구 방향
  - Transformer를 이미지, 오디오, 비디오 등 다른 modality로 확장
  - local / restricted attention 연구
  - 생성 과정의 sequential 의존성 감소 연구
- 한계점
  - 매우 긴 시퀀스를 처리할 때는 계산 비용이 크게 증가하게 됨($O(n^2)$)
  - 기계 번역 말고도 다양한 작업에 대한 일반화 성능 연구 필요

🧐

- RNN이나 CNN과 같은 기존 구조를 개선하는 방식이 아니라, attention만으로 새로운 구조를 설계했다는 점이 매우 인상적이었다.
- 이후 자연어 처리 분야의 많은 모델들이 Transformer 구조를 기반으로 발전했다는 점에서 딥러닝 연구의 중요한 전환점이 된 논문이라고 생각한다.
- Transformer 구조가 매우 강력하지만 모델 크기와 계산 비용이 큰 편이기 때문에 실제 적용에서의 효율적인 모델 설계가 중요할 것 같다.
- 대량의 데이터와 큰 모델 규모를 전제로 성능이 향상되는 경향이 있어, 데이터가 부족한 환경에서는 활용이 어려울 수 있다는 점이 아쉽게 느껴졌다.

교재 개념정리

🔖<(파이토치 트랜스포머를 활용한) 자연어 처리와 컴퓨터 비전 심층학습>

- 6장. 순환신경망 (p.304 ~ 335) / 7장. 트랜스포머 (p.357 ~ 387)

순환 신경망

- **순환 신경망(Recurrent Neural Network, RNN)**
  - 순서가 있는 **연속적인 데이터(Sequence data)**를 처리하는 데 적합한 구조
  - 각 **시점(Time step)**의 데이터가 이전 시점의 데이터와 독립적이지 않다는 특성
- 연속형 데이터
  - 특정 시점에서의 데이터가 이전 시험의 영향을 받는 데이터
  - 자연어 데이터는 연속적인 데이터의 일종
  - 문장 안에서 단어들이 순서대로 등장하므로, 각 단어는 이전에 등장한 단어의 영향을 받아 해당 문장의 의미를 형성

    - 금요일이 지나면 _
    - 수요일이 지나면 목요일이고, 목요일이 지나면 금요일, 금요일이 지나면_

    - 주어진 문장들에서 이전 단어들의 패턴과 의미를 고려해 다음에 올 단어를 유추할 수 있음

  → 자연어는 한 단어가 이전의 단어들과 상호작용하여 문맥을 이루고 의미를 형성

  ⇒ 연속형 데이터의 특성 가짐(t번째 단어는 t-1번째까지의 단어에 영향을 받아 결정)

  - 긴 문장일수록 앞선 단어들과 뒤따르는 단어들 사이에 강한 **상관관계(Correlation)** 존재

    → 이러한 문맥과 상호작용을 모델링해 정확한 의미 파악이 필요

순환 신경망

- 연속적인 데이터를 처리하기 위해 개발된 인공 신경망의 한 종류
- 이전에 처리한 데이터를 다음 단계에 활용, 현재 입력 데이터와 함께 모델 내부에서 과거의 상태를 기억해 현재 상태를 예측하는 데 사용
- 시계열 데이터, 자연어 처리, 음성 인식 및 기타 시퀀스 데이터와 같은 도메인에서 사용
  - 길이가 가변적, 순서에 따라 의미 있는 데이터에 적합한 구조
- 연속형 데이터를 순서대로 입력받아 처리하며 각 시점마다 **은닉 상태(Hidden state)**의 형태로 저장
  - **셀(Cell)**
    - 각 시점의 데이터를 입력으로 받아 은닉 상태와 출력값을 계산하는 노드
    - 이전 시점의 은닉 상태 h_{t-1}을 입력으로 받아 현재 시점의 은닉 상태 h_t를 계산

      ⇒ 이런 재귀적 특징으로 인해 순환 신경망이라 불림

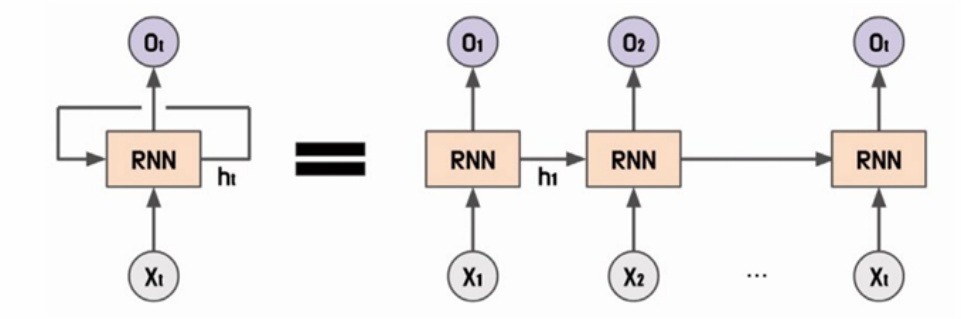

그림 6.12 순환 신경망의 셀

- 순환 신경망은 각 시점 t에서 현재 입력값 x_i와 이전 시점 $t-1$의 은닉 상태 h_{i-1}를 이용해 현재 시점의 은닉 상태 h_i와 출력값 y_i를 계산

수식 6.11 순환 신경망의 은닉 상태

$$h_t = \sigma_h(h_{t-1}, x_t)$$
$$h_t = \sigma_h(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

  - σ_h는 순환 신경망의 은닉 상태를 계산하기 위한 활성화 함수
  - 이전 시점 $t-1$의 은닉 상태 h_{i-1}과 입력값 x_i를 입력받아 현재 시점의 은닉 상태 h_i를 계산
    - 이때 σ_h는 가중치(W)와 편향(b)를 이용해 계산

        - W_{hh}는 이전 시점의 은닉 상태 h_{i-1}에 대한 가중치, W_{zh}는 입력값 x_i에 대한 가중치, b_h는 은닉 상태 h_i의 편향 의미

수식 6.12 순환 신경망의 출력값

$$y_t = \sigma_y(h_t)$$
$$y_t = \sigma_y(W_{hy}h_t + b_y)$$

    - σ_h는 순환 신경망의 출력값을 계산하기 위한 활성화 함수
    - 현재 시점의 은닉 상태 h_i를 입력으로 받아 출력값 y_i를 계산
        - 가중치(W)와 편향(b)를 이용해 계산
        - W_{hy}는 현재 시점의 은닉 상태 h_i에 대한 가중치, b_y는 출력값 y_i의 편향 의미
- 순환 신경망의 출력값은 이전 시점의 정보를 현재 시점에서 활용해 입력 시퀀스의 패턴을 파악하고 출력값을 예측하므로 연속형 데이터를 처리 가능
- 단순 순환 구조부터 일대다 구조, 다대일 구조, 다대다 구조 등이 있음

일대다 구조

- **일대다 구조(One-to-Many)**
    - 하나의 입력 시퀀스에 대해 여러 개의 출력값을 생성하는 순환 신경망 구조
    - 일대다 구조를 사용하여 문장을 입력으로 받고, 문장에서 각 단어의 품사를 예측하는 작업 할 수 있음
    - 이미지 데이터를 입력으로 받으면 이미지에 대한 설명을 출력하는 **이미지 캡셔닝(Image Captioning)** 모델이 됨
        - 입력 시퀀스는 이미지, 출력값은 이미지에 대한 설명 문장들
    - 일대다 구조를 구현하기 위해서는 출력 시퀀스의 길이 미리 알고 있어야 함

  ⇒ 입력 시퀀스를 처리하면서 시퀀스의 정보를 활용해 출력 시퀀스의 길이 예측하는 모델 함께 구현

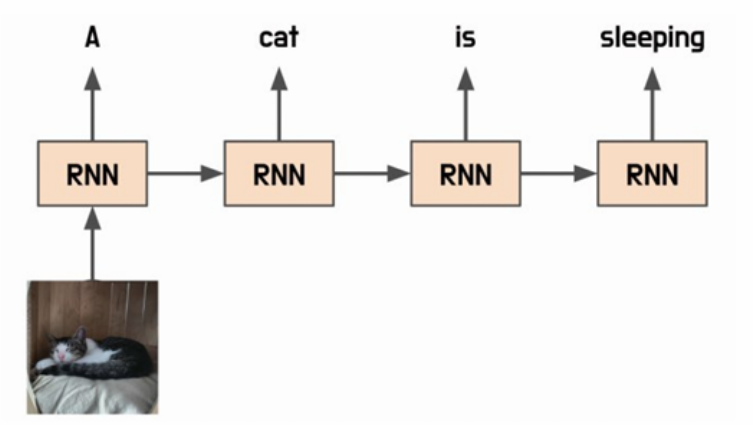

그림 6.13 순환 신경망의 일대다 구조

다대일 구조

- **다대일 구조(Many-to-One)**
  - 여러 개의 입력 시퀀스에 대해 하나의 출력값을 생성하는 순환 신경망 구조
  - **감성 분류(Sentiment Analysis)** 분야에서는 다대일 구조 사용하여 특정 문장의 감정(긍정, 부정) 예측하는 작업을 할 수 있음
    - 입력 시퀀스는 문장, 출력값은 해당 문장의 감정(긍정, 부정)
  - 입력 시퀀스가 어떤 범주에 속하는지는 구분하는 문장 분류, 두 문장 간의 관계를 추론하는 **자연어 추론(Natural Language Inference)** 등에도 적용

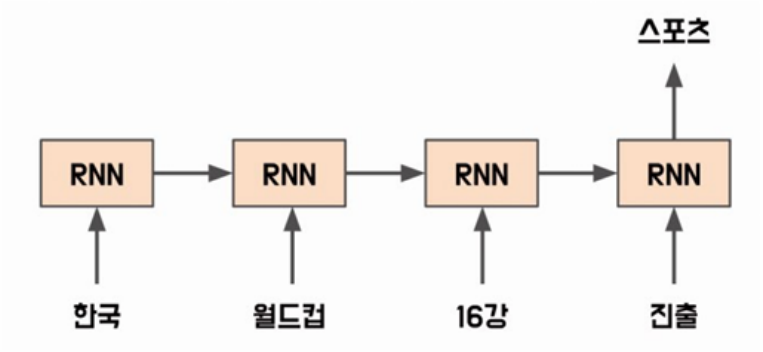

그림 6.14 순환 신경망의 다대일 구조

다대다 구조

- **다대다 구조(Many-to-Many)**
  - 입력 시퀀스와 출력 시퀀스의 길이가 여러 개인 경우에 사용되는 순환 신경망 구조
  - 번역기, 음성 인식기 등에서 사용

- 입력 시퀀스와 출력 시퀀스의 길이가 서로 다른 경우 패딩을 추가하거나 잘라내는 등 전처리 과정이 수행됨
- **시퀀스-시퀀스(Seq2Seq)** 구조로 이뤄짐
  - 입력 시퀀스를 처리하는 **인코더(Encoder)**, 출력 시퀀스를 생성하는 **디코더(Decoder)**로 구성
  - 인코더는 입력 시퀀스를 처리해 고정 크기의 벡터를 출력, 디코더는 이 벡터를 입력으로 받아 출력 시퀀스를 생성

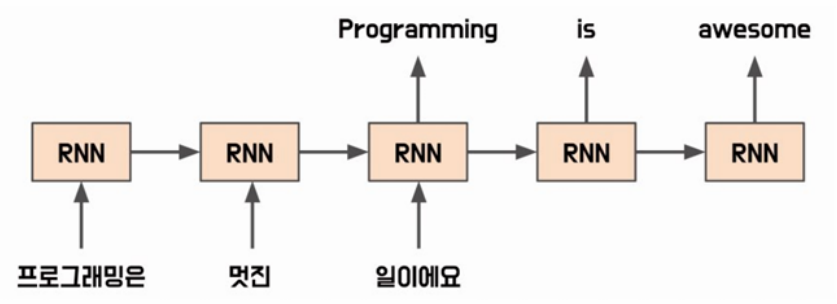

그림 6.15 순환 신경망의 다대다 구조

양방향 순환 신경망

- **양방향 순환 신경망(Bidirectional Recurrent Neural Network, BiRNN)**
  - 기본적인 순환 신경망에서 시간 방향을 양방향으로 처리할 수 있도록 고안된 방식
  - 이전 시점($t-1$)의 은닉 상태뿐 아니라 이후 시점($t+1$)의 은닉 상태도 함께 이용
  - t 시점 이후의 데이터도 t 시점의 데이터를 예측하는데 사용

    → 입력 데이터를 순방향으로 처리하는 것만이 아닌, 역방향으로 거꾸로 읽어 들여 처리하는 방식
  - 양방향적 정보를 모두 고려하여 현재 시점의 출력값을 계산

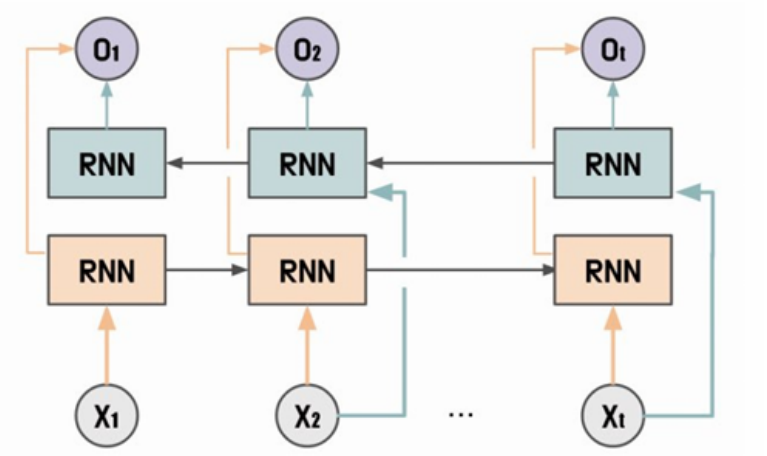

그림 6.16 양방향 순환 신경망 구조

다중 순환 신경망

- **다중 순환 신경망(Stacked Recurrent Neural Network)**
  - 여러 개의 순환 신경망을 연결하여 구성한 모델로, 각 순환 신경망이 서로 다른 정보를 처리하도록 설계되어 있음
  - 여러 개의 순환 신경망 층으로 구성되며, 각 층에서의 출력값은 다음 층으로 전달되어 처리됨
  - 데이터의 다양한 특징 추출할 수 있어 성능 향상, 층이 깊어질수록 더 복잡한 패턴 학습 가능
  - 층이 많아질수록 학습 시간 오래 걸리고 기울기 소실 문제 발생할 가능성 높아지는 단점

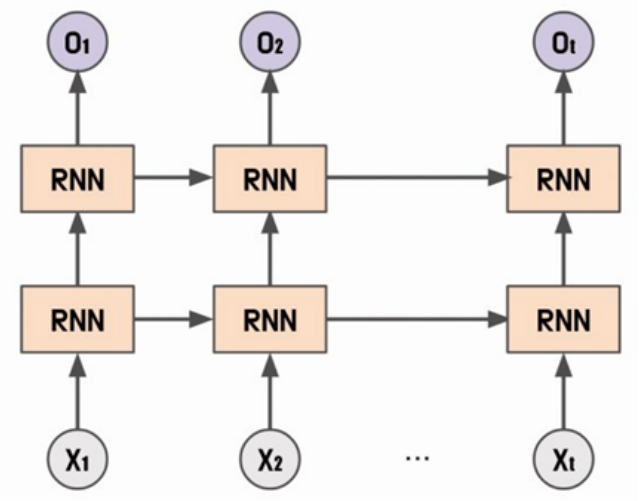

그림 6.17 다중 순환 신경망 구조

트랜스포머

- **트랜스포머(Transformer)**
  - 2017년 소개된 신경망 아키텍처
  - 병렬로 입력 시퀀스를 처리, 긴 시퀀스의 경우 순환 신경망 모델보다 더 빠르고 효율적으로 처리

  ⇒ **순차 처리(Sequential Processing)**나 **반복 연결(Recurrent Connections)**에 의존하지 않고 입력 토큰 간의 관계를 직접 처리하고 이해할 수 있도록 하는 **셀프 어텐션(Self-Attention)**을 기반으로 함

  → 모델이 재귀나 합성곱 연산 없이 입력 토큰 간의 관계 직접 모델링 가능
- 대용량 데이터세트에서 효율적으로 학습 가능 → 데이터의 양 많은 기계 번역 같은 작업에 적합
- 언어 모델링, 텍스트 분류, 광범위한 자연어 처리 작업, 기계번역, 언어 모델링, 텍스트 요약 작업
- 트랜스포머 기반 모델들은 **오토 인코딩(Auto-Encoding), 자기 회귀(Auto Regressive)** 방식으로 학습
- **오토 인코딩**
  - 랜덤하게 문장의 일부를 빈칸 토큰으로 만들고 해당 빈칸에 어떤 단어가 적절할지 예측하는 작업 수행
  - 예측되는 토큰의 양 옆에 있는 토큰들을 참조하기 때문에 양방향 구조 → **인코더(Encoder)**
- 트랜스포머 구조

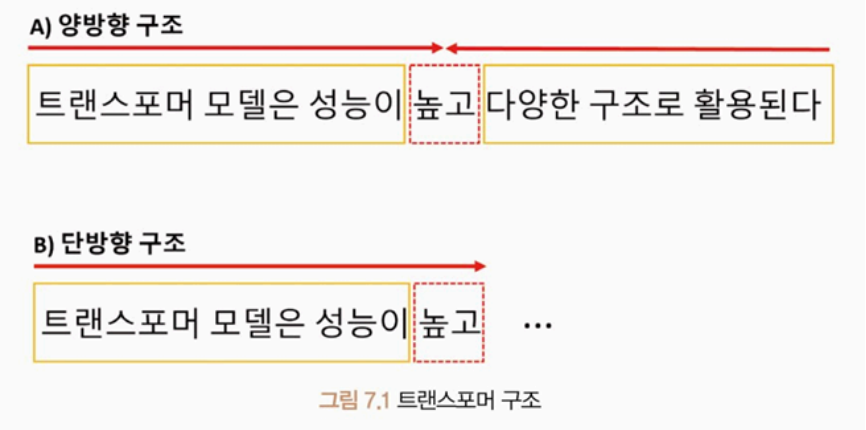

그림 7.1 트랜스포머 구조

- A) ‘트랜스포머 모델은 성능이’, ‘다양한 구조로 활용된다’: 입력, ‘높고’: 출력 양방향 구조

  → 반면 자기 회귀 방식은 이전 단어들이 주어졌을 때 다음 단어가 무엇인지 맞히는 작업을 수행

- B) ‘트랜스포머 모델은 성능이’: 입력, ‘높고’: 출력 단방향 구조

  → 예측되는 토큰의 왼쪽에 있는 토큰들만 참조하기 때문에 단방향 구조 → **디코더(Decoder)**

⇒ 소개하는 트랜스포머 모델들은 양방향성 구조의 인코더(오토 인코딩) 또는 단방향성 디코더(자기 회귀)를 사용하는 공통적 특징

표 7.1 트랜스포머 모델 구조

모델	학습구조	학습방법	학습 방향성
BERT	인코더	오토 인코딩	양방향
GPT	디코더	자기 회귀	단방향
BART	인코더+디코더	오토 인코딩+자기 회귀	양방향+단반향
ELECTRA	인코더+판별기	오토 인코딩+대체 토큰 탐지	양방향
T5	인코더+디코더	오토 인코딩+자기 회귀+다양한 자연어 처리 작업을 학습	양방향

Transformer

- **트랜스포머(Transformer)**
  - 기계 번역, 챗봇, 음성 인식 등 다양한 자연어 처리 분야에서 성과 내는 딥러닝 모델
  - **어텐션 메커니즘(Attention Mechanism)**만을 사용하여 시퀀스 임베딩을 표현
- 트랜스포머의 어텐션 메커니즘
  - 인코더와 디코더 간의 상호작용 → 입력 시퀀스의 중요한 부분에 초점을 맞추어 문맥을 이해하고 적절한 출력을 생성
  - 인코더 - 입력 시퀀스를 임베딩해 고차원 벡터로 변환
  - 디코더 - 인코더의 출력을 입력으로 받아 출력 시퀀스를 생성
  - 어텐션 메커니즘은 인코더와 디코더 단어 사이의 상관관계를 계산하여 중요한 정보에 집중

    → 입력 시퀀스의 각 단어가 출력 시퀀스의 어떤 단어와 관련이 있는지를 파악하여 번역이나 요약문 생성과 관련된 작업 등을 수행

- 트랜스포머 모델은 기존의 순환 신경망 기반 모델보다 학습 속도가 빠르고, 병렬 처리가 가능해 대규모 데이터세트에서 높은 성능 보임
- 임베딩 과정에서 문장의 전체 정보를 고려 → 문장의 길이 길어지더라도 성능이 유지되는 장점
- 트랜스포머 모델 구조

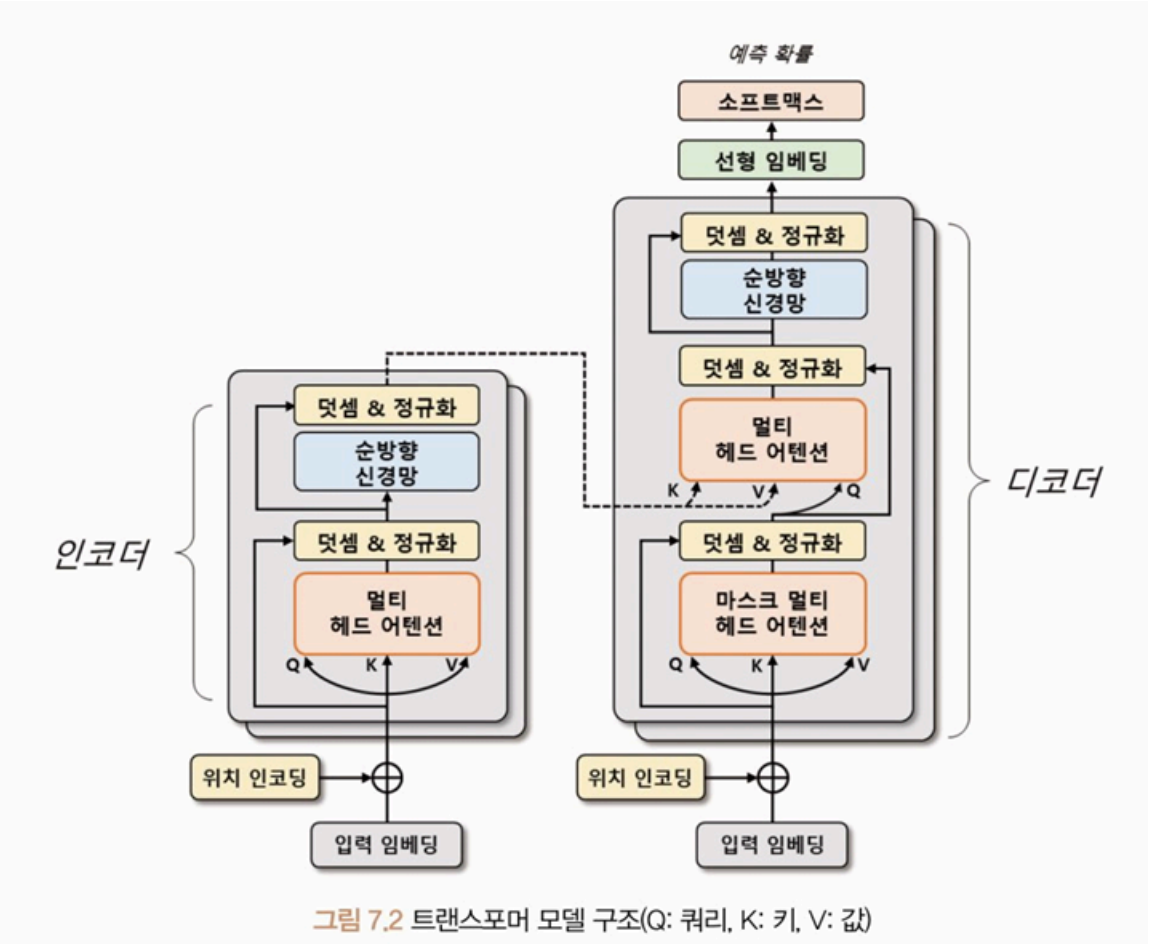

그림 7.2 트랜스포머 모델 구조(Q: 쿼리, K: 키, V: 값)

  - 인코더와 디코더는 두 부분으로 구성되어 있으며, 각각 N개의 **트랜스포머 블록(Transformer Block)**으로 구성

    → 이 블록은 **멀티 헤드 어텐션(Multi-Head Attention)**과 순방향 신경망으로 이루어짐
  - 멀티 헤드 어텐션
    - 입력 시퀀스에서 **쿼리(Query), 키(Key), 값(Value)** 벡터를 정의해 입력 시퀀스들의 관계를 **셀프 어텐션(Self-Attention)**하는 벡터 표현 방법
    - 쿼리와 각 키의 유사도를 계산, 해당 유사도를 가중치로 사용하여 값 벡터를 합산

        - 계산된 어텐션 행렬은 입력 시퀀스 각 단어의 임베딩 벡터를 대체

    ⇒ 입력 시퀀스의 단어 사이의 상호작용을 고려해 임베딩 벡터를 갱신함

    - 순방향 신경망
        - 산출된 임베딩 벡터를 더욱 고도화하기 위해 사용
        - 여러 개의 선형 계층으로 구성, 앞선 순방향 신경망의 구조와 동일하게 입력 벡터에 가중치를 곱하고 편향을 더하여 활성화 함수를 적용
        - 학습된 가중치들은 입력 시퀀스의 각 단어의 의미를 잘 파악할 수 있는 방식으로 갱신

- 트랜스포머에서는 입력 시퀀스 데이터를 **소스(Source)**와 **타깃(Target)** 데이터로 나눠 처리

  Ex) 영어를 한글로 번역하는 경우 - 생성하는 언어인 한국어를 타깃, 참조하는 언어인 영어를 소스 데이터로 정의

    - 인코더는 소스 시퀀스 데이터를 **위치 인코딩(Positional Encoding)**된 입력 임베딩으로 트랜스포머 블록의 출력 벡터를 생성
        - 출력 벡터는 입력 시퀀스 데이터의 관계를 잘 표현할 수 있게 구성됨
    - 디코더는 **마스크 멀티 헤드 어텐션(Masked Multi-Head Attention)**을 사용해 타깃 시퀀스 데이터를 순차적으로 생성
        - 이때 디코더 입력 시퀀스들의 관계를 고도화하기 위해 인코더의 출력 벡터 정보를 참조
    - 최종적으로 생성된 디코더 출력 벡터는 선형 임베딩으로 재표현되어 이미지나 자연어 모델에 활용됨

입력 임베딩과 위치 인코딩

- 트랜스포머 모델은 입력 시퀀스를 병렬 구조로 처리하기 때문에 단어의 순서 정보를 제공 X

  → 위치 정보를 임베딩 벡터에 추가해 단어의 순서 정보를 모델에 반영해야 함

  ⇒ 인코딩 방식 사용

- 위치 인코딩
    - 입력 시퀀스의 순서 정보를 모델에 전달하는 방법
    - 각 단어의 위치 정보를 나타내는 벡터를 더하여 임베딩 벡터에 위치 정보를 반영
    - 위치 인코딩 벡터를 추가함으로써 모델은 단어의 순서 정보를 학습할 수 있게 됨

- sin 함수, cos 함수를 사용해 임베딩 벡터와 위치 정보가 결합된 최종 입력 벡터를 생성
  - 각 토큰의 위치를 각도로 표현해 sin 함수, cos 함수로 위치 인코딩 벡터를 계산
  - 토큰의 위치마다 동일한 임베딩 벡터를 사용 X, 각 토큰의 위치 정보를 모델이 학습 가능
- 위치 인코딩 계산 방식

수식 7.1 위치 인코딩

$$PE(pos, 2i) = \sin(pos/10000^{2i/d_{model}})$$
$$PE(pos, 2i+1) = \cos(pos/10000^{2i/d_{model}})$$

  - pos는 입력 시퀀스에서 해당 단어의 위치, i는 임베딩 벡터의 차원 인덱스
  - 차원의 인덱스가 짝수면 sin, 홀수면 cos 함수 수식 적용
  - 각도 정보로 변환하기 위한 스케일링 인자 → 각 위치에 대한 주기적인 신호 생성

특수 토큰

- 트랜스포머는 단어 토큰 이외의 특수 토큰을 활용하여 문장 표현
- 특수 토큰은 입력 시퀀스의 시작과 끝 나타내거나 **마스킹(Masking)**영역으로 사용
- 모델이 입력 시퀀스의 시작과 끝 인식할 수 있게 하며, 마스킹을 통해 일부 입력 무시 가능

  Ex) 번역 모델 - 디코더의 입력 시퀀스에서 현재 위치 이후의 토큰을 마스크해 이전 토큰만을 참조

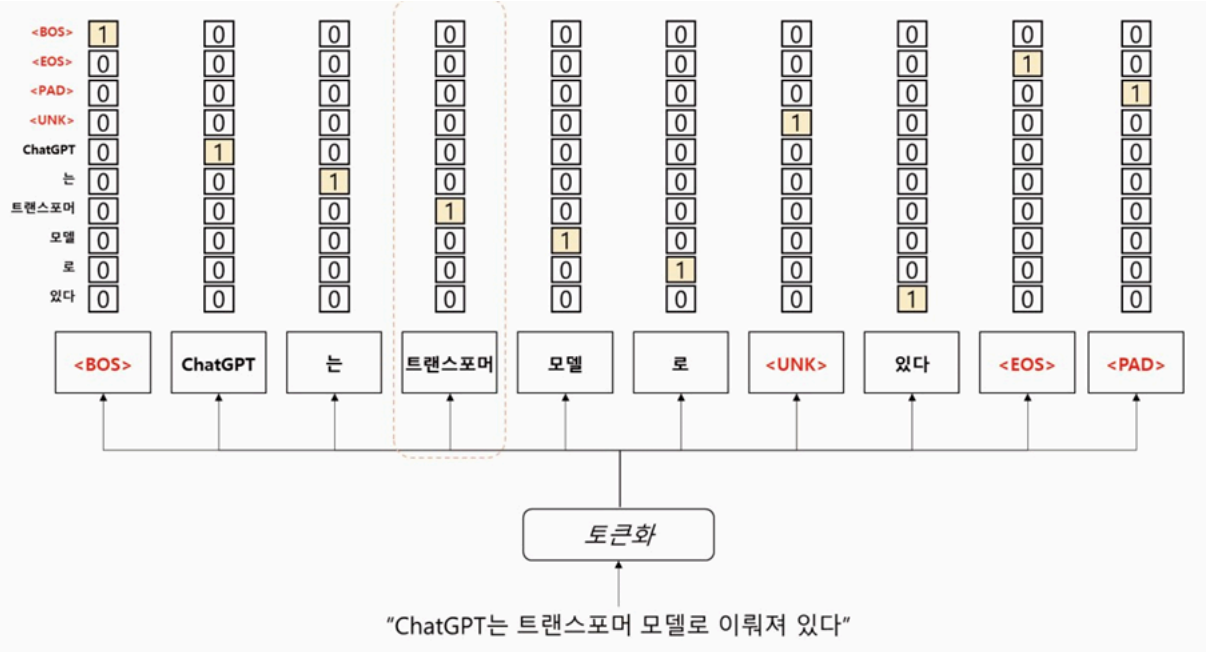

그림 7.3 특수 토큰을 활용한 문장 토큰 배열

- **BOS(Beginnig of Sentence), EOS(End of Sentence), UNK(Unknown), PAD(Padding)** 토큰

  → 모두 특수 토큰으로 자연어 처리에서 일반적으로 사용됨

  - BOS 토큰: 문장의 시작 나타냄
  - EOS 토큰: 문장의 종료 나타냄

    → 문장의 시작과 끝 명확히 하기 위해 사용
  - UNK 토큰: 어휘 사전에 없는 단어로, 모르는 단어를 의미

    → 모델이 이전에 본 적 없는 단어 처리할 때 사용
  - PAD 토큰: 모든 문장을 일정한 길이로 맞추기 위해 사용

    → 짧은 문장의 빈 공간을 채울 때 사용
- 생성된 문장 토큰 배열을 어휘 사전에 등장하는 위치에 원-핫 인코딩으로 표현

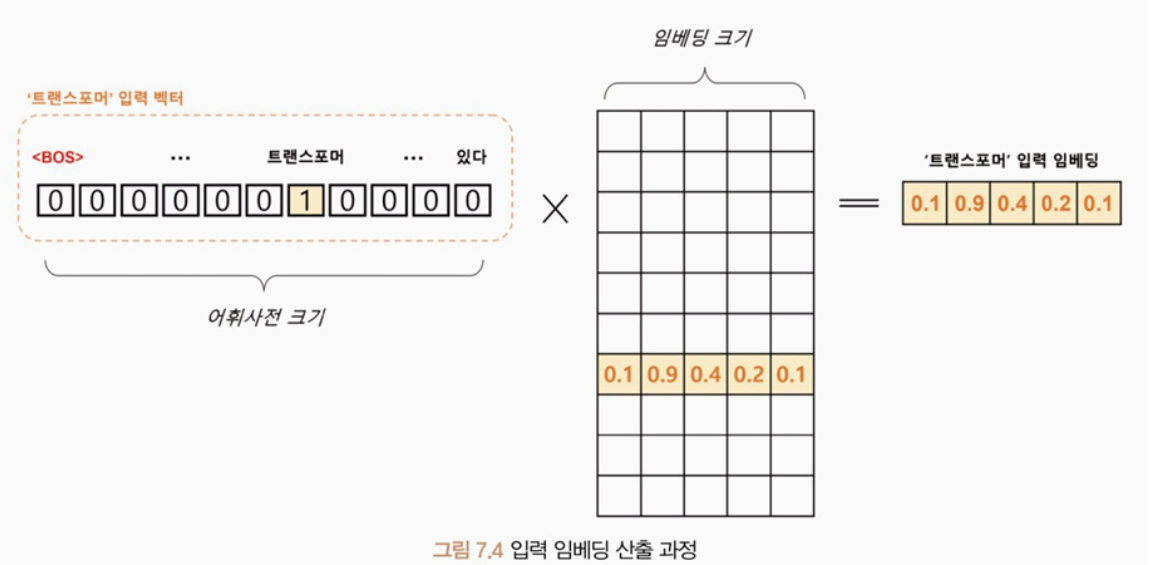

그림 7.4 입력 임베딩 산출 과정

- 입력 임베딩으로 변환되는 과정은 Word2Vec과 동일
- 어휘 사전의 크기를 V, 입력 임베딩 차원을 d라 했을 때 '트랜스포머'의 원-핫 벡터는 [1, V] 크기
- 원-핫 벡터는 임베딩 행렬 [V, d]에 의해 [1, d] 크기의 벡터로 변환됨

⇒ N개의 문장이 최대 S개의 토큰 길이를 가질 때 [N, S, V] 크기의 원-핫 벡터 텐서는 [N, S, d] 크기의 임베딩 텐서로 변환됨

- 임베딩 텐서는 입력으로 사용되며, 트랜스포머의 모든 계층에서 공유하는 텐서

트랜스포머 인코더

- 입력 시퀀스를 받아 여러 개의 계층으로 구성된 인코더 계층을 거쳐 연산을 수행
- 각 인코더 계층은 멀티 헤드 어텐션과 순방향 신경망으로 구성, 입력 데이터에 대한 정보를 추출하고 다음 계층으로 전달
  - 인코더 계층에서 위치 정보를 반영하기 위해 위치 임베딩 벡터를 입력 벡터에 더해줌
  - 산출된 인코더 계층의 출력은 디코더 계층으로 전달됨

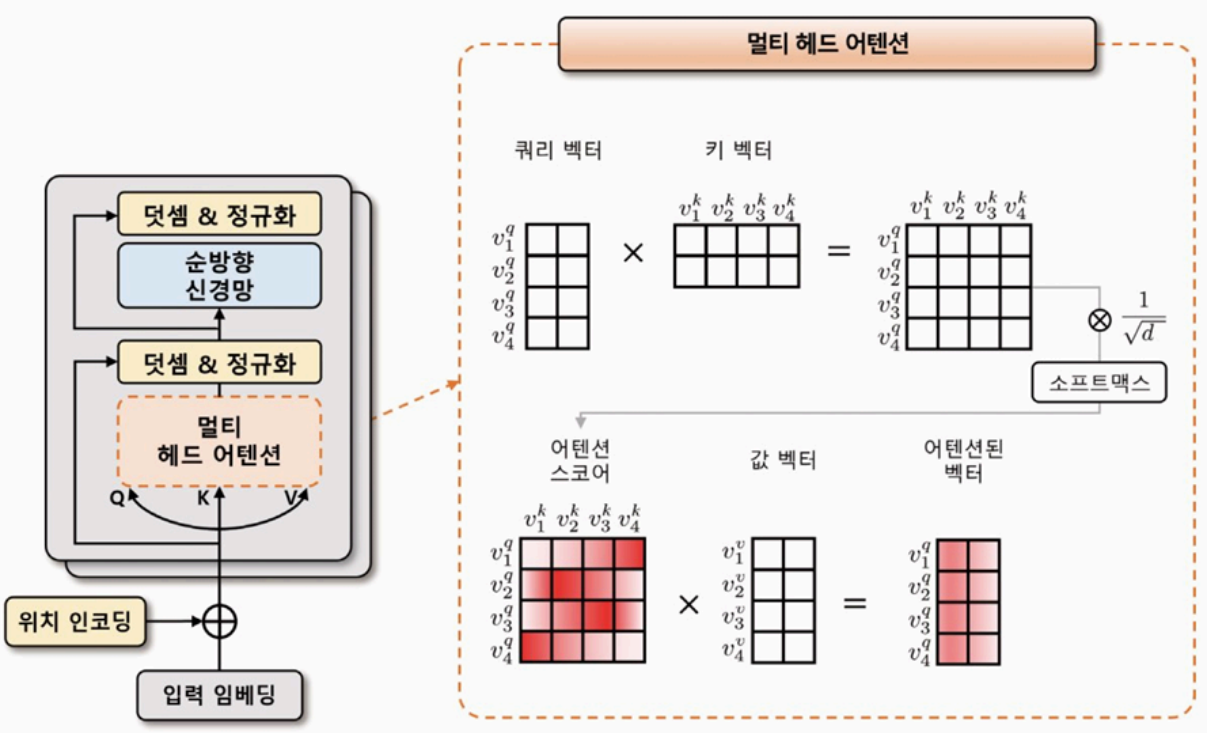

그림 7.5 트랜스포머 인코더 블록 연산 과정

- 트랜스포머 인코더는 위치 인코딩이 적용된 소스 데이터의 입력 임베딩을 입력받음
  - 멀티 헤드 어텐션 단계에서 입력 텐서 차원이 [N, S, d]라면 입력 임베딩은 선형 변환을 통해 3개의 임베딩 벡터를 생성
  - 생성된 3개의 벡터를 각각 쿼리(Q), 키(K), 값(V) 벡터라고 정의
    - **쿼리 벡터**(v^q)
      - 현재 시점에서 참조하고자 하는 정보의 위치 나타내는 벡터
      - 인코더의 각 시점마다 생성
      - 현재 시점에서 질문이 되는 벡터를 의미하며, 이 벡터를 기준으로 다른 시점의 정보를 참조
    - **키 벡터**(v^k)
      - 쿼리 벡터와 비교되는 대상으로, 쿼리 벡터를 제외한 입력 시퀀스에서 탐색되는 벡터
      - 인코더의 각 시점에서 생성
    - **값 벡터**(v^v)
      - 쿼리 벡터와 키 벡터로 생성된 어텐션 스코어를 얼마나 반영할지 설정하는 가중치 역할

    - 쿼리, 키, 값 벡터는 초기에 비슷한 값 가지지만 모델 학습을 통해 의도한 의미의 값을 갖게 됨
    - 쿼리와 키 벡터의 연관성은 내적 연산으로 [N, S, S] 어텐션 스코어 맵을 사용
- 쿼리, 키 벡터를 사용한 어텐션 스코어 계산식

수식 7.2 어텐션 스코어 계산식

$$score(v^q, v^k) = \mathrm{softmax}\left(\frac{(v^q)^T \cdot v^k}{\sqrt{d}}\right)$$

    - 쿼리, 키, 값 벡터를 내적해 어텐션 스코어를 구하고 이 스코어값에 $\sqrt{d}$(벡터 차원의 제곱근) 만큼 나눠 보정

      → 보정값은 벡터 차원이 커질 때 스코어값이 같이 커지는 문제를 완화
    - 보정된 어텐션 스코어를 소프트맥스 함수를 이용해 확률적으로 재표현하고, 이를 값 벡터와 내적하여 셀프 어텐션 된 벡터를 생성

      → 이 과정 반복해 셀프 어텐션된 스코어 맵 생성
- **멀티 헤드(Multi-Head)**는 이러한 셀프 어텐션을 여러 번 수행해 여러 개의 헤드 만듦
    - 각각의 헤드가 독립적으로 어텐션을 수행하고 그 결과를 합침
    - 입력받는 [N, S, d] 텐서에 k개의 셀프 어텐션 벡터를 생성하고자 한다면 헤드에 대한 차원 축을 생성해 [N, k, S, d/k] 텐서 형태를 구성 - k개의 셀프 어텐션된 [N, S, d/k] 텐서를 의미
    - 생성된 k개의 셀프 어텐션 벡터는 임베딩 차원 축으로 다시 **병합(concatenation)** 되어 [N, S, d]의 형태로 출력
- 덧셈 & 정규화
    - 멀티 헤드 어텐션을 통과하기 이전의 입력값 [N, S, d] 텐서와 통과한 이후의 출력값 [N, S, d] 텐서를 더함 → 학습 시 발생하는 기울기 소실 완화
    - 이 값에 임베딩 차원 축으로 정규화하는 계층 정규화를 적용
- 순방향 신경망
    - 선형 임베딩과 ReLU로 이뤄진 인공신경망 또는 1차원 합성곱이 사용됨
    - 이어서 다시 덧셈 & 정규화 과정 수행
- 트랜스포머 인코더는 여러 개의 트랜스포머 인코더 블록으로 구성

- 이전 블록에서 출력된 벡터는 다음 블록의 입력으로 전달되어 인코더 블록을 통과하면서 입력 시퀀스의 정보가 추상화됨
- 마지막 인코더 블록에서 출력된 벡터는 디코더에서 사용 → 디코더의 멀티 헤드 어텐션 모듈에서 참조되는 키, 값 벡터로 활용

⇒ 이런 방식으로 인코더와 디코더는 서로 정보를 공유

트랜스포머 디코더

- 위치 인코딩이 적용된 타깃 데이터의 입력 임베딩을 입력받음
- 디코더의 입력 임베딩에 위치 정보를 추가함으로써 디코더가 입력 시퀀스의 순서 정보를 학습 가능
- 인코더의 멀티 헤드 어텐션 모듈은 **인과성(Casuality)**를 반영한 마스크 멀티 헤드 어텐션 모듈로 대체됨
  - 어텐션 스코어 맵을 계산할 때 첫 번째 쿼리 벡터는 첫 번째 키 벡터만, 두 번째 쿼리 벡터는 첫 번째와 두 번째 키 벡터를 바라보게 마스크를 씌움

    → 셀프 어텐션에서 현재 위치 이전의 단어들만 참조할 수 있게 되며, 인과성이 보장됨
  - 마스크 영역에 수치적으로 굉장히 작은 값인 -inf 마스크를 더해줌으로써 해당 영역의 어텐션 스코어값이 0에 가까워짐
  - 소프트맥스 계산 시 해당 영역의 어텐션 가중치가 0이 되므로, 마스크 영역 이전에 있는 입력 토큰들에 대한 정보를 참조하지 않게 됨

⇒ 마스크 멀티 헤드 어텐션 모듈은 인코더의 멀티 헤드 어텐션 모듈과 유사하지만, 인과성을 보장하면서 셀프 어텐션을 수행할 수 있게 됨

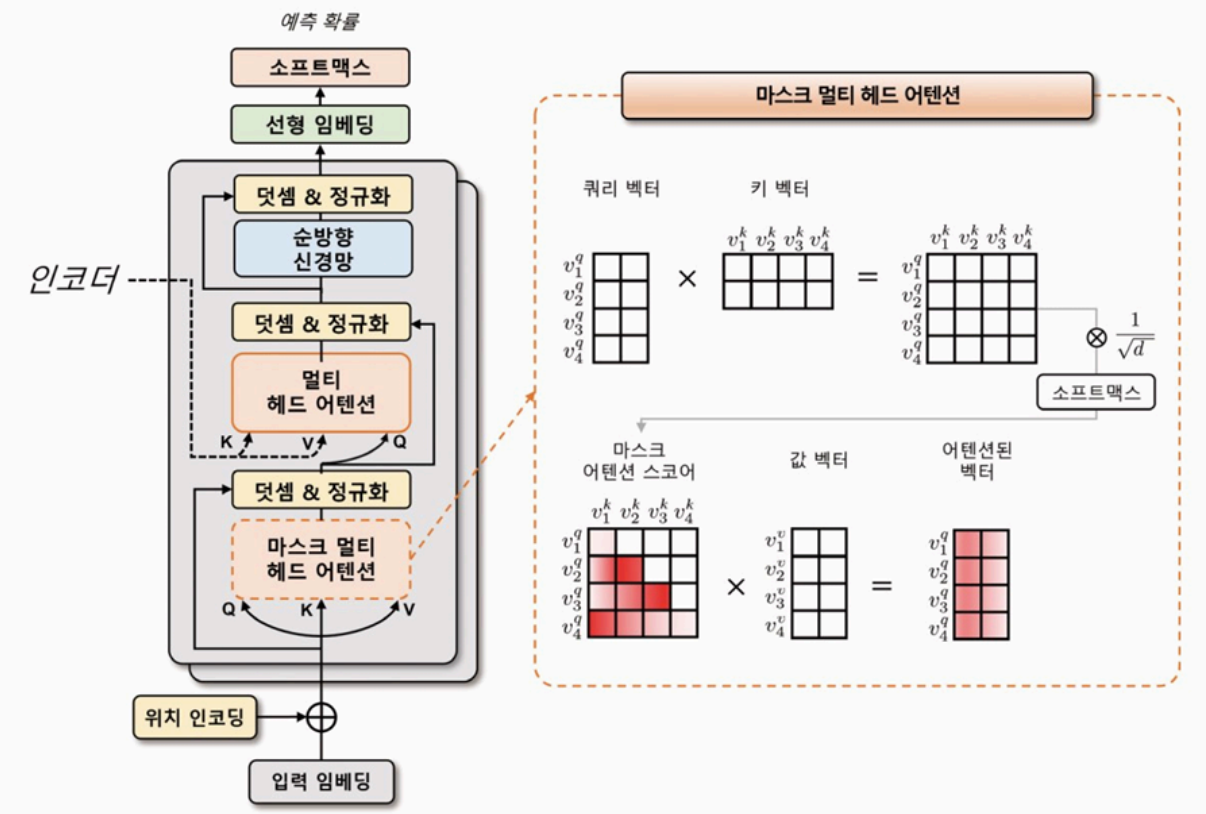

그림 7.6 트랜스포머 디코더 블록 연산 과정

- 디코더의 멀티 헤드 어텐션에서는 타깃 데이터가 쿼리 벡터로 사용되며, 인코더의 소스 데이터가 키와 값 벡터로 사용됨

  → 쿼리 벡터는 타깃 데이터의 위치 정보를 포함한 입력 임베딩과 위치 인코딩을 더한 벡터가 됨

  - 이후 쿼리, 키, 값 벡터를 이용해 어텐션 스코어 맵을 계산하고, 소프트맥스 함수를 적용하여 어텐션 가중치를 구함
  - 최종으로 어텐션 가중치와 값 벡터를 가중합하여 멀티 헤드 어텐션의 출력 벡터를 얻을 수 있음
- 디코더에서는 셀프 어텐션을 방지하기 위해 마스킹을 적용
- 디코더는 여러 개의 트랜스포머 디코더 블록으로 구성
  - 이전 트랜스포머 디코더 블록의 산출물은 다음 트랜스포머 디코더 블록의 입력으로 전달됨

    → 이전 시점에서의 정보를 현재 시점에서 활용할 수 있게 됨
  - 마지막 디코더 블록의 출력 텐서 [N, S, d]에 선형 변환 및 소프트맥스 함수를 적용해 각 타깃 시퀀스 위치마다 예측 확률을 계산할 수 있게 됨
- 디코더는 타깃 데이터를 추론할 때 토큰 또는 단어를 순차적으로 생성시키는 모델

- 입력 토큰을 순차적으로 나타내는 방법은 빈 값을 의미하는 PAD 토큰을 사용하는 것

- ‘ChatGPT는 트랜스포머 모델로 이뤄져 있다’라는 문장을 추론할 때 디코더의 입력과 출력 예시

표 7.2 인과성을 고려한 디코더 입력과 출력 예시

추론 순서	디코더 입력	디코더 출력
1	[BOS], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], ‘ChatGPT’, [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]
2	[BOS], ‘ChatGPT’, [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], ‘ChatGPT’, ‘는’, [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]
3	[BOS], ‘ChatGPT’, ‘는’, [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], ‘ChatGPT’, ‘는’, ‘트랜스포머’, [PAD], [PAD], [PAD], [PAD], [PAD]
4	[BOS], ‘ChatGPT’, ‘는’, ‘트랜스포머’, [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], ‘ChatGPT’, ‘는’, ‘트랜스포머’, ‘모델로’, [PAD], [PAD], [PAD], [PAD]
5	[BOS], ‘ChatGPT’, ‘는’, ‘트랜스포머’, ‘모델로’, [PAD], [PAD], [PAD], [PAD]	[BOS], ‘ChatGPT’, ‘는’, ‘트랜스포머’, ‘모델로’, ‘이뤄져’, [PAD], [PAD], [PAD]
6	[BOS], ‘ChatGPT’, ‘는’, ‘트랜스포머’, ‘모델로’, ‘이뤄져’, [PAD], [PAD], [PAD]	[BOS], ‘ChatGPT’, ‘는’, ‘트랜스포머’, ‘모델로’, ‘이뤄져’, ‘있다’, [PAD], [PAD]
7	[BOS], ‘ChatGPT’, ‘는’, ‘트랜스포머’, ‘모델로’, ‘이뤄져’, ‘있다’, [PAD], [PAD]	[BOS], ‘ChatGPT’, ‘는’, ‘트랜스포머’, ‘모델로’, ‘이뤄져’, ‘있다’, [EOS], [PAD]